

Data Lakehouse и табличные форматы

Parquet

Delta / Iceberg / Hudi

Bronze / Silver / Gold

Лекция 6

Где мы в курсе

HDFS: как хранить большие данные распределённо

Spark: как обрабатывать данные распределённо

Lakehouse: как управлять таблицами поверх файлового хранилища

Цель: сделать данные воспроизводимыми, проверяемыми и удобными для потребителей

Почему обычного Data Lake мало

Нет транзакций

можно сделать overwrite
(перезаписать) таблицу

Нет версий

статистика за вчера
сегодня ночью overwrite

Schema drift

producer поменял колонку
все упало

Маленькие файлы

тысячи маленьких файлов
замедляют чтение

Нет проверки качества

качество обнаруживается
слишком поздно

Идея Lakehouse: оставить дешёвое файловое хранение, но добавить свойства управляемых таблиц

Медальонная архитектура

Bronze

сырой append-only слой
минимум
преобразований
аудит и
воспроизводимость

Silver

очищенные сущности
dedup, типы, ключи
quality checks

Gold

витрины под задачи
BI, ML, отчёты, SLA

Формат файла ≠ формат таблицы

File format

Parquet, ORC, Avro, CSV

Отвечает на вопросы:

- как байты лежат на диске
- как хранится схема
- как читать колонки

Table format

Delta Lake, Apache Iceberg, Apache Hudi

Отвечает на вопросы:

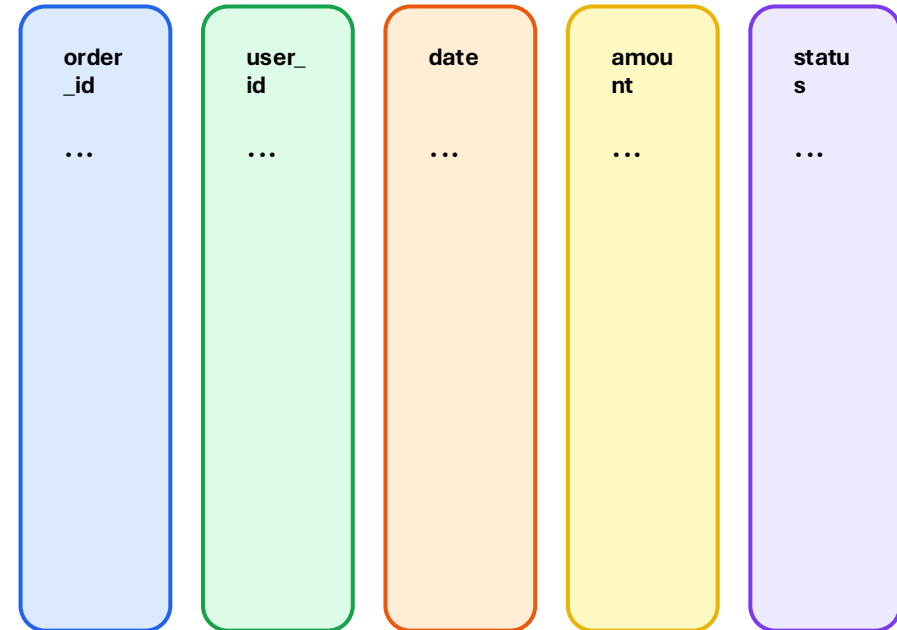
- какие файлы входят в таблицу
- какая версия актуальна
- как делать commit/rollback

Parquet: базовый кирпич Lakehouse

Колоночное хранение: читаем только нужные колонки

Predicate pushdown: часть фильтров можно применить на уровне чтения

Parquet — отличный формат файла, но сам по себе не даёт транзакции и версии



Partitioning: ускорение через layout

```
orders/  
  dt=2026-04-01/  
    part-000.parquet  
  dt=2026-04-02/  
    part-000.parquet  
  dt=2026-04-03/  
    part-000.parquet
```

Хороший фильтр

where dt = '2026-04-02'
читаем одну
директорию

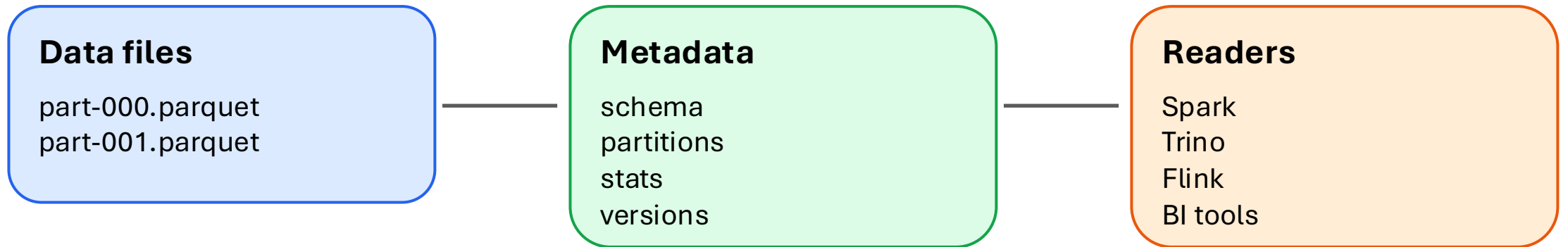
Плохой фильтр

where amount > 100
читаем все партиции

Правило выбора partition columns

Низкая/средняя уникальность ключа, частые
фильтры, стабильный ключ.
Дата обычно хороша; user_id почти всегда опасен.

Metadata: сердце table format



Metadata говорит:

- Какая схема у данных
- Какие есть партиции
- Какие есть версии этих данных

Все это помогает сервисам чтения (Readers) считывать данные эффективно

MERGE / UPSERT / CDC

Append

добавить новые
строки

Update

заменить актуальное
состояние ключа

Delete

удалить или пометить
запись

Merge

insert/update/delete
по ключу

Без table format upsert часто превращается в тяжёлый full overwrite

С table format это управляемая операция с новой версией

ACID поверх файлов

Atomicity

Атомарность

либо выполнено
полностью
либо не выполнено
никак

Consistency

Согласованность

схема и
ограничения
(constraints) не
нарушены

Isolation

Изолированность

транзакции не
затирают друг друга

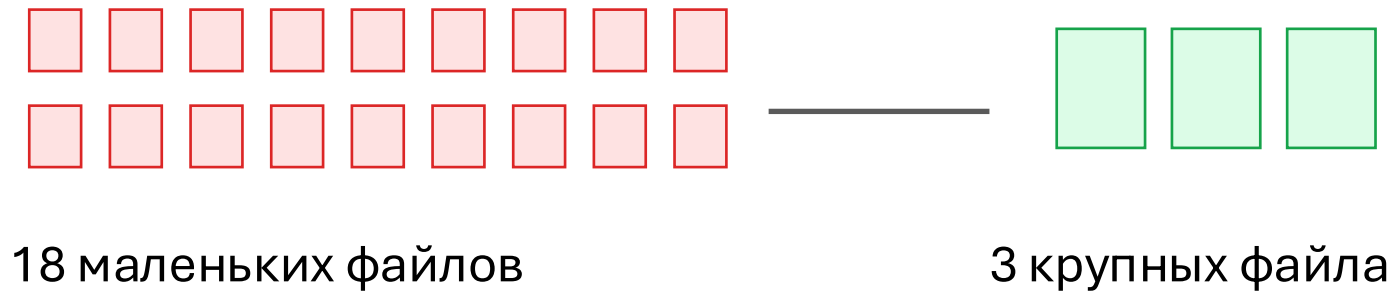
Durability

Надежность

commit переживает
сбой системы

Commit protocol: сначала пишем новые файлы, затем публикуем новую metadata-версию.

Маленькие файлы и компактность (compaction)



Что делать

1. Контролировать coalesce/repartition перед записью
2. Не делать partitionBy по слишком разнообразным ключам

Schema evolution и enforcement

Evolution Эволюция

Высокая гибкость, низкая надежность

Разрешённые изменения:

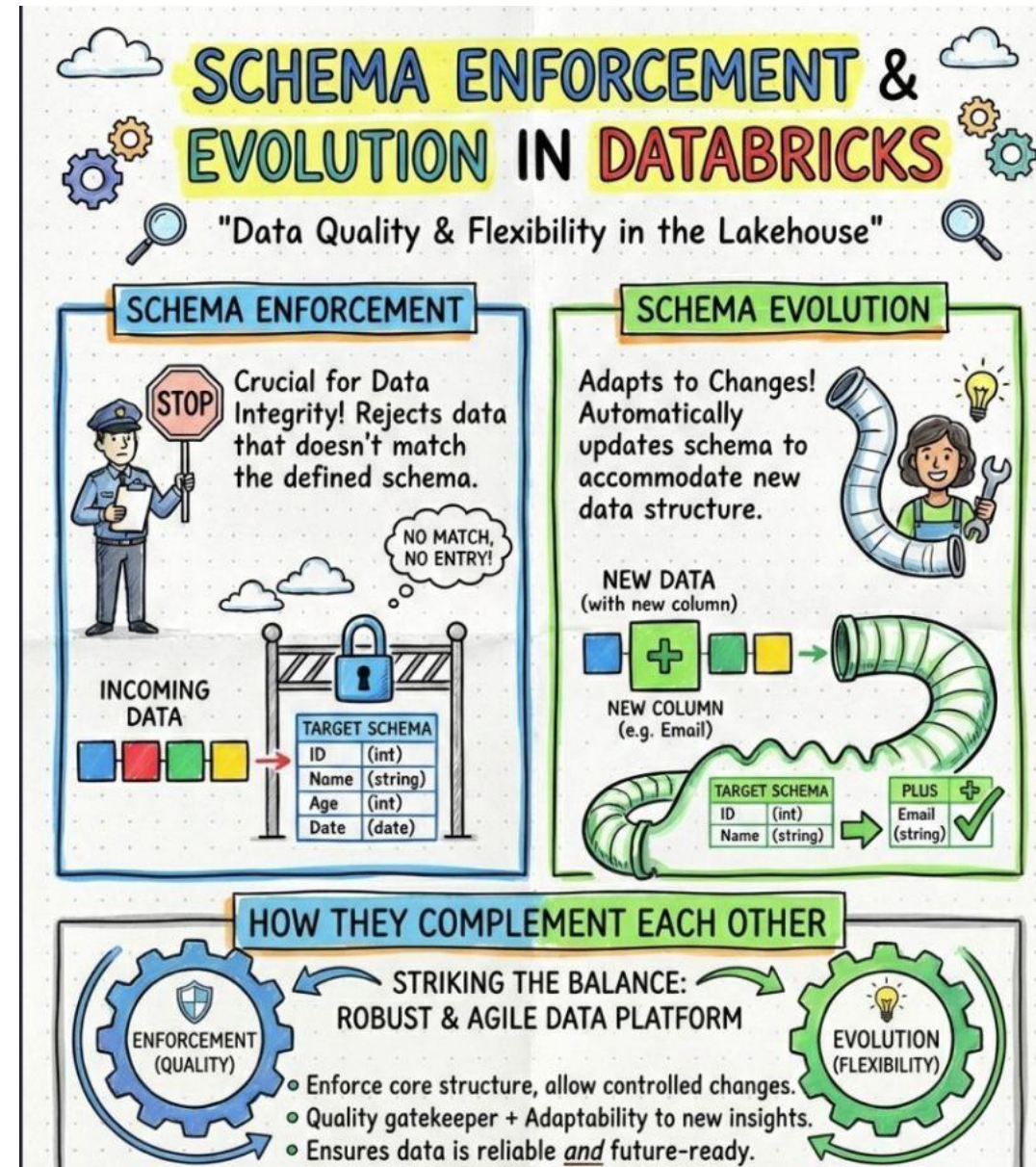
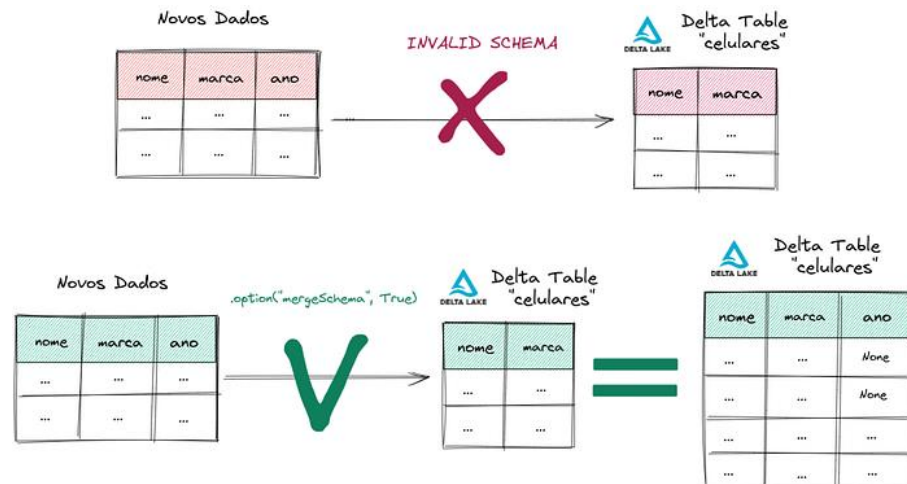
- добавить nullable column
- расширить тип

Enforcement Принуждение

Низкая гибкость, высокая надежность

Нельзя:

- менять схему
- вставлять данные другой схемы



Time travel и reproducibility



Data quality рядом с данными

Completeness

Полнота

обязательные поля не null

Uniqueness

Уникальность

ключи уникальны

Validity

Валидность

значения в допустимых
диапазонах

Referential integrity

Ссылочная целостность

foreign keys существуют

Freshness

Актуальность

данные не устарели

Reconciliation

Согласованность

сверка сумм между слоями

Проверки должны блокировать публикацию плохой витрины.

Delta Lake / Iceberg / Hudi



Выбор зависит от стека: Spark-only или multi-engine, CDC (Change Data Capture), cloud, команды эксплуатации.

Delta Lake

table/
 part-000.parquet
 part-001.parquet
 _delta_log/
 000000000000000000000000.json
 000000000000000000000001.json

000000000000000000000002.checkpoint.parquet

Что хранит log

add/remove file actions, schema, protocol, statistics, commit info

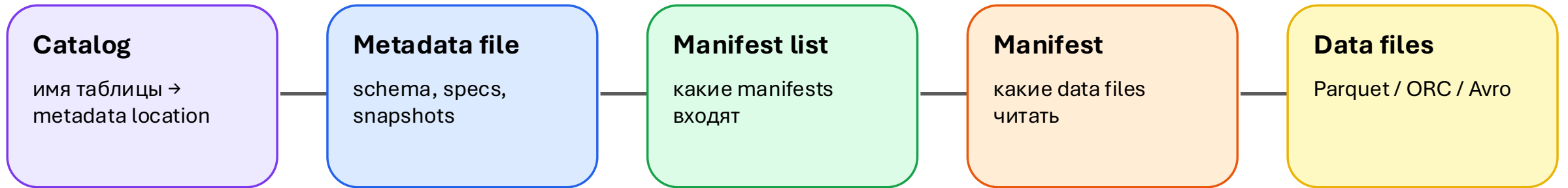
Что получает пользователь

ACID, time travel, MERGE, UPDATE/DELETE, schema enforcement/evolution

Когда удобно

Spark-first ETL, медальонная архитектура

Apache Iceberg



Скрытое партиционирование: Iceberg сам занимается партиционированием по колонкам

Partition evolution: схему можно менять со временем без переписывания старых данных

Силен в multi-engine сценариях: Spark, Trino, Flink, Presto, Hive-compatible catalogs

Apache Hudi: данные меняются

Перезапись

(copy-on-write)

Обновили заказ — переписали Parquet-файл.

Читать просто: готовая версия.

Писать дороже: файл может быть большим.

Слияние

(merge-on-read)

Изменения пишутся в журналы.

Запись быстрее.

Чтение собирает базовый файл + изменения.

Изменения

(incremental reads)

“Что изменилось после прошлого запуска?”

Следующий ETL не читает всю таблицу заново.

Главная мысль: Hudi полезен, когда в lake приезжают изменения старых записей, а не только новые строки.

Apache Hudi: история изменений и ключи

История

(timeline)

Журнал: что и когда изменилось.

По нему Hudi видит текущую версию.

И отдает изменения после нужного момента.

Ключ записи

(record key)

order_id: тот же заказ или новый?

Поле свежести (precombine field) выбирает последнюю версию.

Старое событие не должно перезаписать новое.

Режимы

Перезапись: читать проще, писать тяжелее.

Слияние при чтении: быстрее принимать изменения.

Выбор: что важнее — чтение или запись.

Пример: заказ создали, оплатили, отменили. Hudi помогает собрать из потока событий актуальное состояние таблицы.

Как выбирать: Delta vs Iceberg vs Hudi

Если стек Spark/Databricks

Delta Lake

Если нужен multi-engine lakehouse

Apache Iceberg

Если главный сценарий
CDC/upsert/incremental reads

Apache Hudi

Вопросы выбора: какие движки читают таблицы, как часто идут обновления, нужен ли streaming, какие SLA.

Типовой production pipeline

